LABORATORY MANUAL

Cloud Native Application and DevOps

Deploying and Scaling a Containerized Web Application on Kubernetes Using Docker Desktop

Duration: 1–2 Hours

Platform: Docker Desktop + Kubernetes
Environment: Windows 10/11 + PowerShell

1. Experiment Information

Item	Details
Experiment Title	Deploying and Scaling a Containerized Web Application on Kubernetes Using Docker Desktop
Course	Cloud Native Application and DevOps
Duration	1–2 Hours
Platform	Docker Desktop with Kubernetes
Language / Tools	Dockerfile, YAML, kubectl, PowerShell
Difficulty	Beginner–Intermediate

2. Learning Objectives

- Enable and verify Kubernetes in Docker Desktop.
- Build a Docker image for a simple web application.
- Deploy the image using a Kubernetes Deployment.
- Understand Pods, Deployments, ReplicaSets and Services.
- Expose an application using a Kubernetes Service.
- Scale an application up and down using replicas.
- Demonstrate Kubernetes self-healing.
- Perform a simple rolling update and rollback.
- Use essential kubectl commands.

3. Background

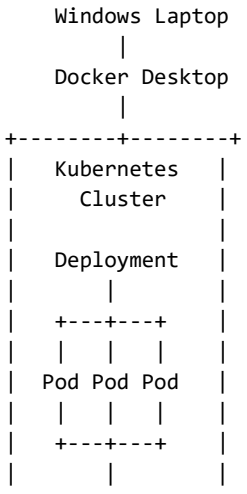
Kubernetes is a container orchestration platform that manages containerized workloads. It automates deployment, scaling, networking, service discovery, self-healing and rolling updates.

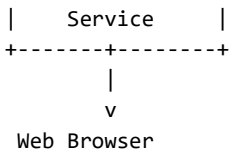
```
Docker:
Dockerfile → Image → Container

Kubernetes:
Docker Image → Deployment → Pods → Service → Users
```

In this experiment, Docker Desktop provides the local Kubernetes environment. Students create a Docker image, deploy it to Kubernetes, expose it through a Service, scale replicas and test self-healing.

4. Lab Architecture





5. Requirements

Software	Purpose
Docker Desktop	Container runtime and local Kubernetes
PowerShell	Execute commands
VS Code / Text Editor	Create Dockerfile and YAML
Web Browser	Test the application

6. Prerequisites

- Basic Docker concepts
- Basic Linux/PowerShell commands
- Basic YAML syntax
- Basic cloud-native concepts

7. Step 1 — Verify Docker Desktop

Start Docker Desktop and wait until Docker Engine is running.

```
docker --version

docker info

docker run hello-world
```

Expected: Docker successfully runs the hello-world container.

8. Step 2 — Enable Kubernetes in Docker Desktop

Open Docker Desktop and use its Kubernetes section to create or enable a local Kubernetes cluster. Choose the simple local cluster option suitable for the class.

After Kubernetes becomes Ready, verify:

```
kubectl config use-context docker-desktop

kubectl get nodes

kubectl cluster-info
```

Expected: the Docker Desktop Kubernetes node is in Ready state.

9. Step 3 — Create Project Directory

```
mkdir C:\kubernetes-demo
cd C:\kubernetes-demo
```

Create:

```
Dockerfile
index.html
deployment.yaml
service.yaml
```

10. Step 4 — Create the Web Application

Create index.html:

```
<!DOCTYPE html>
<html>
<head>
    <title>Kubernetes Demo</title>
</head>
<body>
    <h1>Cloud Native Application</h1>
    <h2>Running on Kubernetes</h2>
    <p>Docker + Kubernetes using Docker Desktop</p>
</body>
</html>
```

11. Step 5 — Create Dockerfile

```
FROM nginx:alpine

COPY index.html /usr/share/nginx/html/index.html

EXPOSE 80
```

The Dockerfile builds an Nginx image containing the custom web page.

12. Step 6 — Build the Docker Image

```
docker build -t kubernetes-demo:v1 .

docker images
```

Expected image:

```
kubernetes-demo    v1
```

13. Step 7 — Test the Container Before Kubernetes

```
docker run -d --name demo-test -p 8080:80 kubernetes-demo:v1
```

Open:

```
http://localhost:8080
```

After successful testing, remove the test container:

```
docker rm -f demo-test
```

14. Step 8 — Create Kubernetes Deployment

Create deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment

metadata:
  name: kubernetes-demo

spec:
  replicas: 3

  selector:
    matchLabels:
      app: kubernetes-demo

  template:
    metadata:
      labels:
        app: kubernetes-demo

    spec:
      containers:
        - name: web
          image: kubernetes-demo:v1
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 80
```

Key idea: replicas: 3 tells the Deployment to maintain three Pods.

15. Step 9 — Deploy to Kubernetes

```
kubectl apply -f deployment.yaml

kubectl get deployments

kubectl get pods
```

Expected: three Pods should be Running.

16. Understanding Pod, Deployment and ReplicaSet

```
Deployment
    |
    v
ReplicaSet
    |
    +---- Pod 1
    |
    +---- Pod 2
    |
    +---- Pod 3
          |
          v
       Container
```

The Deployment declares the desired number and version of Pods. The ReplicaSet helps maintain the requested number of Pods.

17. Step 10 — Create a Kubernetes Service

Create service.yaml:

```
apiVersion: v1
kind: Service

metadata:
  name: kubernetes-demo-service

spec:
  selector:
    app: kubernetes-demo

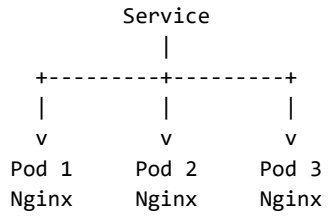
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80

  type: NodePort

kubectl apply -f service.yaml

kubectl get services
```

18. Understanding the Service



The Service provides a stable endpoint and selects Pods using the label app: kubernetes-demo.

19. Step 11 — Access the Application

For a simple Docker Desktop laboratory, use port-forwarding:

```
kubectl port-forward service/kubernetes-demo-service 8080:80
```

Open in a browser:

```
http://localhost:8080
```

Leave the port-forward command running while testing.

20. Step 12 — Scale the Application

Current replicas:

```
kubectl get deployment kubernetes-demo
```

Scale to five:

```
kubectl scale deployment kubernetes-demo --replicas=5
```

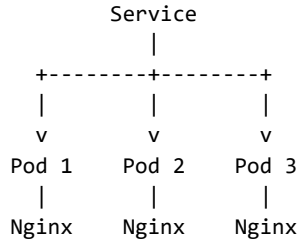
```
kubectl get pods
```

Scale back to two:

```
kubectl scale deployment kubernetes-demo --replicas=2

kubectl get pods
```

21. Scaling Architecture




```
Example:
replicas = 3 → 3 Pods
replicas = 5 → 5 Pods
```

22. Step 13 — Demonstrate Self-Healing

Make sure three replicas are running:

```
kubectl scale deployment kubernetes-demo --replicas=3

kubectl get pods
```

Choose one Pod and delete it:

```
kubectl delete pod <pod-name>
```

Immediately check:

```
kubectl get pods
```

Expected: Kubernetes creates a replacement Pod so that the desired replica count is restored.

23. Self-Healing Diagram

```
Desired state = 3 Pods

Pod 1     Pod 2     Pod 3
             X
          Pod 2 fails
             |
             v
       Kubernetes detects
       desired != current
             |
             v
          New Pod 4

Pod 1     Pod 3     Pod 4
```

24. Step 14 — View Pod Details

```
kubectl describe pod <pod-name>
```

Observe: image, container, ports, labels, node, conditions and events.

25. Step 15 — View Logs

```
kubectl logs <pod-name>
```

Use this command to investigate application/container output.

26. Step 16 — Rolling Update

Change index.html:

```
<h1>Cloud Native Application v2</h1>
```

Build a new image:

```
docker build -t kubernetes-demo:v2 .
```

Modify deployment.yaml:

```
image: kubernetes-demo:v2
```

Apply the new version:

```
kubectl apply -f deployment.yaml

kubectl rollout status deployment/kubernetes-demo
```

A rolling update replaces old Pods with new Pods according to the Deployment update strategy.

27. Step 17 — Rollback

```
kubectl rollout history deployment/kubernetes-demo

kubectl rollout undo deployment/kubernetes-demo

kubectl rollout status deployment/kubernetes-demo
```

28. Essential kubectl Commands

Command	Purpose
kubectl get nodes	List cluster nodes
kubectl get pods	List Pods
kubectl get deployments	List Deployments
kubectl get services	List Services
kubectl apply -f file.yaml	Create/update resources
kubectl describe pod <name>	Detailed Pod information
kubectl logs <name>	View container logs
kubectl scale deployment <name> --replicas=N	Change desired replicas
kubectl delete pod <name>	Delete a Pod for self-healing demo
kubectl rollout status deployment/<name>	Check rollout
kubectl rollout undo deployment/<name>	Rollback

kubectl delete -f file.yaml	Delete resources defined in a file

29. Practical Observation Table

Activity	Command	Observation
Docker verification	docker info	_____
Kubernetes node	kubectl get nodes	_____
Build image	docker build ...	_____
Deployment	kubectl apply -f deployment.yaml	_____
Pods	kubectl get pods	_____
Service	kubectl get services	_____
Browser test	localhost:8080	_____
Scale up	replicas=5	_____
Scale down	replicas=2	_____
Self-healing	kubectl delete pod ...	_____
Rolling update	kubectl apply -f deployment.yaml	_____
Rollback	kubectl rollout undo ...	_____

30. Practical Tasks

1. Create and test the Docker image.
2. Deploy the image using a Kubernetes Deployment with three replicas.
3. Expose the Deployment through a Kubernetes Service.
4. Scale the Deployment from 3 to 5 Pods.
5. Scale it down to 2 Pods.
6. Delete one Pod and demonstrate self-healing.
7. Create version 2 of the application and perform a rolling update.
8. Perform a rollback to the previous version.
9. Delete all resources after the experiment.

31. Troubleshooting

Problem	Possible Cause	Solution
kubectl not recognized	kubectl unavailable/PATH issue	Use Docker Desktop Kubernetes tools or install kubectl and open a new terminal
No nodes found	Kubernetes not enabled or wrong context	Check Docker Desktop Kubernetes and run kubectl config use-context docker-desktop
ImagePullBackOff	Image not available to cluster	Ensure the local Kubernetes setup can access the Docker Desktop image store; otherwise load/publish the image using the cluster method
Pod Pending	Resource/scheduling issue	Run kubectl describe pod <name> and inspect Events
Service not reachable	Service/Pod selector or port issue	Check kubectl get svc and kubectl describe service
Port already in use	Local port conflict	Use another local port with kubectl port-forward
Nginx page not changed	Old image still running	Use a new image tag and update the Deployment

32. Security and Good Practices

- Use trusted base images where possible.
- Pin image versions/tags rather than relying on latest in production.
- Do not store secrets in Dockerfiles or Git repositories.
- Use Kubernetes Secrets for sensitive configuration, with appropriate protection.
- Expose only required ports.
- Use resource requests and limits for production workloads.
- Review container images for vulnerabilities before production deployment.
- Clean up local resources after the practical.

33. Student Submission Checklist

☐ Screenshot of Docker Desktop with Kubernetes running

☐ Dockerfile

☐ Screenshot of docker build

☐ Screenshot of kubectl get nodes

☐ deployment.yaml

☐ service.yaml

☐ Screenshot of Pods running

☐ Screenshot of Service

☐ Browser screenshot of application

☐ Screenshot of scaling from 3 to 5 Pods

☐ Screenshot of self-healing demonstration

☐ Screenshot of rolling update

☐ Screenshot of rollback

☐ Completed observation table

☐ Viva answers

☐ Cleanup screenshot/output

34. Viva Questions

10. What is Kubernetes?
11. What is the difference between Docker and Kubernetes?
12. What is a Pod?
13. What is a Deployment?
14. What is a ReplicaSet?

15. What is a Kubernetes Service?
16. Why are Pods not normally treated as permanent endpoints?
17. What does replicas: 3 mean?
18. How does Kubernetes provide self-healing?
19. What happens when a Pod is deleted?
20. What is a rolling update?
21. How do you roll back a Deployment?
22. What is kubectl?
23. Why is imagePullPolicy: IfNotPresent useful in a local Docker Desktop lab?
24. How would you move this application from Docker Desktop Kubernetes to a cloud Kubernetes service?

35. Connection with Cloud-Native DevOps

```
Dockerfile
    |
    v
Docker Image
    |
    v
Kubernetes Deployment
    |
    +---- Pods
    |
    +---- Service
    |
    +---- Scaling
    |
    +---- Self-Healing
    |
    +---- Rolling Updates
    |
    v
CI/CD Pipeline
```

This practical follows naturally after containerization and Docker Compose. The next logical topics are Kubernetes ConfigMaps/Secrets, persistent storage, Ingress, autoscaling, Helm and CI/CD deployment.

36. Conclusion

Students deployed a Dockerized web application on Kubernetes running inside Docker Desktop. They created a Deployment and Service, verified Pods, exposed the application locally, scaled replicas, demonstrated self-healing, and performed a rolling update and rollback. The exercise establishes the core Kubernetes workflow needed for cloud-native application deployment and DevOps automation.

37. Instructor Notes

- Verify that all student machines have enough RAM allocated to Docker Desktop.
- Enable Kubernetes before the class or allocate setup time at the beginning.
- Use one simple Nginx application for the first demonstration.
- Ask students to predict what Kubernetes will do before running scale/delete commands.
- Connect the self-healing and scaling activities to the earlier Docker scaling laboratory.
- Require cleanup at the end of the practical.

38. Reference Resources

- Docker Desktop Kubernetes — https://docs.docker.com/desktop/use-desktop/kubernetes/
- Kubernetes Documentation — https://kubernetes.io/docs/
- Kubernetes Concepts — https://kubernetes.io/docs/concepts/
- kubectl Reference — https://kubernetes.io/docs/reference/kubectl/